

STATA Tutorial

(Updated on 01/19/2025)

Some of the tutorial are for your future reference (such as your research) but not for this class. We will only cover what we need for this class.

The latest version is Stata 18. But the contents here should be useful for any version after Stata 7.

Introduction

This tutorial is designed to give you a quick look at some basic tasks you may need to do in Stata. What it does cover is:

1. Overview of Stata.
2. Creating and using "log" and "do" files.
3. Data management.
4. File management.
5. Basic data description.
6. Basics in regression analysis.

There are some other resources you may find useful:

1. Stata manuals
2. Stata Technical Bulletin
3. Stata's web site, www.stata.com
4. Stata's on-line tutorial (the command, 'help tutorial' will show you what to do).

Overview

Stata is an interactive data analysis program which runs on a variety of platforms.

Stata can be used to interactively enter and edit data, and for both simple and complex statistical analysis. Commands can be executed one at a time at the Stata prompt, or groups of commands can be entered into **do-files** which can then be executed.

Data Management

Data management is perhaps the most important part of your work in Stata. It is how you get your data into a format in which you can run your analyses properly. Major mistakes have been the result of poor data management rather than poor analysis.

Reading Data

Before you read your data, you may wish to change the working directory. Suppose that you have all your files in the directory `\coursework\econometrics\econ830`. You can use the following command to change the working directory

```
cd c: \coursework\econometrics\econ830
```

You can use the command `cd` without parameter to find out the current working directory. If you want to see the file names in the current working directory, use `dir`

There are two basic ways of reading data into Stata. "**use**" is for reading data that have been saved in Stata format, "**insheet**" is for spreadsheets saved as "CSV" files from a package such as Excel or some other ascii files.

Use

The Stata **use** command reads data that has been saved in Stata format:

```
use [filename]
```

where *filename* is the name of the Stata file. For example, suppose you have a Stata file named "food.dta." You can read this file with the following command:

```
use food
```

Note that the ".dta" file extension is automatically appended to Stata files. You do not have to include the file extension on the **use** command. Also note that the file should be stored in your working directory. You can also read files from other directories, e.g.:

```
use \home\project\wage
```

If there is current a data set in your memory which you have not saved since last change, use the following command to clear the memory and read the data

```
use \home\project\myfile, clear
```

Insheet

The **insheet** command is very useful in reading in data from a spreadsheet. The first line should have Stata variable names (eight characters or less, not starting with a "special character" or number) and the second line begins the data.

```
insheet using filename.csv
```

For example, you can use the following command to read the food data file in ASCII format:

```
insheet using food.txt
```

How to Save Stata Data Files

You can use the Stata **save** command to save a file in Stata format:

```
save [filename]
```

where *filename* is the name of your Stata file. For example:

```
save myfood
```

will save a Stata file named "myfood.dta" in your working directory. This file can be read in Stata with the **use** command. Note that the ".dta" file extension is automatically appended to Stata files. You do not have to include the file extension on the **use** or **save** commands.

If you already have a Stata file named "myfood.dta" and wish to save an updated version of the file under the same name, then use the Stata **save** command with the **replace** option, as in:

```
save [filename], replace
```

where *filename* is the name of the file you wish to replace, e.g.:

```
save myfood, replace
```

To save an updated version of the active file, you can simply type:

```
save, replace
```

To save your file in some other directory, you can type:

```
save \endless\homework1\food, replace
```

This command will destroy the previous version of your file so use the **replace** option only if you are certain that you will not need the older version of your file. There is no way to retrieve your original file once another file has written over it.

Examining your Data

Describe

Once you have the data in Stata, you will want to make sure that all the variables are there and that they are in the format you need. You can do this with the "**describe**" command. Describe, which can be abbreviated as simply "**d**," will provide basic information about the file and the variables. You don't have to call the data into Stata to be able to describe it, though. The command:

```
d
or
d using food
```

will accomplish this. This can be very useful if the file is too large to fit into memory.

List

It is often useful to just look at the data without doing any kind of analysis. The "**list**" command, abbreviated as "**l**" will let you do this. Simply giving the "**l**" command will display all of the data on your screen, if you specify certain variables in the command, then only those variables will be printed:

```
l
l food income
l in 5/10
l food income in 5/10
```

In Stata for Windows you can have a direct look at your data as if you were working with a spreadsheet (e.g. Excel), by using the "Stata Editor". You enter the editor by clicking on the "Editor" button. You can edit your data by highlighting the cell you want to change, entering the new value for that cell, and clicking on "return". You can edit the variable name (and the variable label) by double clicking on the relevant column.

in and **if** can be used at the end of many commands to apply the command only to certain observations or when certain conditions are met. For example:

```
l food income if income > 100
```

Creating Variables

Stata can store data as either numbers or characters. Stata will allow you to do most analyses only on numeric data. Sometimes, when you use `insheet` or `infile`, a numeric variable gets read in as a character, or string variable. This may be due to a 'space' character or a '.' in one of the cells. Since Stata allows you to do analyses only on numeric variables, you will need to convert them to numeric data.

There are different types of numeric variables - float, binary, double, long and int - the differences among them are simply how much space they take up in the file. In most

cases, you will not need to concern yourself with these differences.

Generate

Often, you will need to create new variables based on the ones you have already. The two most common ways of creating new variables is by using "**generate**" and "**egen**." Here are some examples of `gen`:

```
gen other= income-food
gen cumtot=sum(income)
gen id=_n
```

In the first example, we generate a new variable called "other". In the next example, we create a cumulative sum of total (the value of `cumtot` for this observation is the sum of total for all previous observations). Finally, we create an "id" variable by simply setting it equal to "`_n`", which is Stata's way of numbering the observations in a dataset. This id variable can be very useful when you need to sort your data in different ways (see sorting below). The variable "`_n`" can be used at any time, but you must remember that it refers to the number of the observation in the current order, not the original order.

Extended Generate (egen)

"**egen**", or "extended generate" is useful when you need a new variable that is the mean, median, etc. of another variable, for all observations or for groups of observations. Egen is also useful when you need to simply number groups of observations based on some classification variables. Here are some examples:

```
egen sumvar1 = sum(var1)
egen meanvar1= mean(var1), by(var3)
egen count = count(id), by(company)
egen group = group(month year)
```

In the first example, we simply create a variable whose value for each observation is equal to the sum of `var1` for all observations (all observations will have the same value). The second example shows how to create a variable that is the mean of another variable for each group designated by `var3` (all observations within a group will have the same value). The third example simply counts the number of non-missing `id` values in each company (this basically counts the number of observations you have for each company). The last example simply assigns a number to each of the groups created by the combination of month and year.

There are many functions that can be used with "extended generate". To have a brief summary, type:

```
help egen
```

Variable and Value labels

Now that we've created all these new variables, we'll want some way of keeping track of what each one is and what their values mean. We can do this by creating variable labels and value labels. These labels are not necessary in Stata, they just make the output easier to read. Variable labels correspond to the variable names, whereas value labels correspond to the different values a variable may have. Here's how to create them:

```
label variable food "Weekly expenditure on food"
```

This assigns a label to the variable, food, so whenever the variable name food is displayed on the screen, the description "Weekly expenditure on food" will be displayed as well.

Variable labels must be 31 characters or less. Often, though, it's more helpful to label the values a variable can take so you don't have to memorize them or keep referring to a codebook.

```
label define grp 1 "Male" 2 "Female" 3 "N/A"  
label values gender grp
```

Value labels correspond to the actual number or letters in the data. They are used so that the printout will show "Male," "Female," and "N/A" instead of "1," "2," and "3". First you have to "define" the label, then associate that label with a variable. Each label can be associated with more than one variable, so if there are several "yes/no/maybe" questions, you just have to define one value label and use it for all the questions. You may associate only one value label with a particular variable, however.

You can check the list and content of all value labels by typing:

```
label list
```

Keep, Drop, and Rename

Well, now we've created many new variables and converted some old ones. Since we no longer need all of these variables, we'll want to eliminate some of the ones we don't really need and perhaps rename some of the ones we keep. We can either **keep** the variable we are interested in:

```
keep code date var4
```

or we can **drop** those we don't need, and **rename** some of those we keep:

```
drop var1 var2 var3  
rename code date
```

Here we drop the variables, var1, var2 and var3. Be careful! Once they are dropped, they can't be picked up again unless you clear the data and use it again. You can drop as many variables as you want in one command. The same is true for "**keep**". The second example renames the variable "code" to "date." You can rename only one variable at a time. You can also drop (or keep) observations:

```
drop if size==1
```

or

```
keep if size~=1
```

will drop all observations whose size is equal to 1.

Log and Do Files

Log files

Log and do files are very useful. Logs keep a record of what commands you have issued and their results during your Stata session. Do files are good for long series of commands that may need to be "tweaked" to work properly. They are also necessary to replicate things that you have done on new or modified datasets.

You can create a log file with the command:

```
log using filename, text
```

Where *filename* is any name you wish to give the file. You will find it helpful to use names that will help you to remember what you did during that session. Stata will automatically append an extension of ".log" to the filename. By default, everything displayed on the screen will be recorded in the log file. You can give the file any name you like, but you should use names that will help you remember what analyses you did.

If you work for a long time, this file can get to be very large and have quite a bit of unnecessary output. So, if you are trying different things to "see what happens," you may want to start and stop the logging several times in one session. You can stop the logging with the command:

```
log close
```

If you turn the logging on and off during a session, you may need to use the append and/or replace options:

```
log using filename, append
```

```
log using filename, replace
```

The append option simply adds more information to the file, whereas the replace option erases anything that was already in the file. Be careful! It's often helpful to put comments in the log file to help you remember why you did something. You can do this by simply preceding anything you type with a "*".

But, what happens if you forget to start a log file? There is a way to save what you have done by using the log file and the #review command. The #review command will list the last however many commands you have issued. This command can be used at any time and you can specify any number of commands to retrieve. After some minor editing and changing the extension to ".do", you can run it as a do file. Here's an example:

```
log using iforgot - start a log file
```

```
#review 20 - list the last 20 commands
log close - close the log file
```

The file, iforgot.log, will have the last twenty commands you issued in it. You can then run this file as a do file with a new log started.

Do Files

A "do" file is a set of commands just as you would type them in one-by-one during a regular Stata session. Any command you use in Stata can be part of a do file. Do files are very useful, particularly when you have many commands to issue repeatedly, or to reproduce results with minor or no changes.

You can use any editor you wish to create your do files. In Stata you can click on the Do-file Editor and type your commands, then save it as a do file. For example, you can save your do file as proj1. Stata automatically adds .do as the extension name.

File Management

Once you have all of your variables in the format you want, you'll need to get the entire file in a format that will make it easier to use. You can do this by sorting, appending, merging and collapsing.

Sort

"**sort**" puts the observations in a data set in a specific order. Some procedures require the file to be sorted before it can work. You can sort a file based on more than one variable. One thing you must be careful of, though, is that Stata will randomize the order of the observations within the variables used to sort. This is why creating the id variable mentioned earlier is important. With the id variable, you will always be able to go back to the original order and start over.

```
sort month
sort month year
```


Append

Sometimes, you have more than one file of data which you need to analyze. One case may be that you have two files with the same variables but different observations. The other case is when you have two files with the same observations, but different variables.

Use **append** when you simply want to add more observations, in other words you already have data for 1990, and now you want to include new observations from 1991 for the same variables. For example:

```
use class90
append using class91
```

will add the observations from the file, class91 (what Stata refers to as the "using" dataset), to the end of class90 (what Stata refers to as the "master" dataset). Any variables with different names in the two files will have missing values for the observations from the other dataset.

Merge

If you have two files that have the same observations, but different variables, then you'll want to "merge" them so you can use all of the variables at once. When you merge datasets, you are adding new variables to existing observations rather than adding observations to existing variables. There are two basic kinds of merges, a one-to-one and a match.

A one-to-one merge simply takes the two files and puts them side-by-side, regardless of whether the observations in each dataset are in the same order. A simple one-to-one merge isn't very common, and isn't recommended, even if you think all the observations match. Sometimes there may be a "glitch" that can throw off the order of the data. It's just as easy to do a match-merge, and you'll be able to check that all the observations matched correctly.

There are some "rules" when doing a match-merge. First, each dataset must have a "key" variable by which the observations can be matched - social security number is a good example. Second, both datasets must be sorted by this key variable. You can use more than one key variable if you want, such as month and year. These key variables must be of the same type (string or numeric) in both datasets. By default, if a variable is present in both datasets, then the values in the master dataset will remain unchanged. If the variables in the using dataset have the same names as ones in the master dataset, but represent additional information, then you will need to rename them in one of the datasets before you can merge them.

```
use class90
sort date
merge date using class91
```

In this example we simply match observations using the variable "date" as the key.

Sometimes, you will want to update the data you have, namely, fill in missing values. Using the update option will accomplish this:

```
merge date using class91, update
```

Values in the master dataset will be changed only if they are missing.

In some instances, you may want to replace non-missing values in the master dataset. Use the replace option in addition to the update option to do this:

```
merge date using class91, update replace
```

"**replace**" will not work by itself, it must always be used with update. Stata will not, under any circumstances, change a non-missing value in the master dataset with a missing value from the using dataset. You must use the "**replace**" command discussed earlier to do this.

Stata will automatically create a variable called **_merge** which will indicate the results of the merge. Always check this variable to make sure that you got what you wanted. Here are what the possible values of **_merge** mean:

- 1 = Observations from the master dataset that *did not* match observations from the using dataset.
- 2 = Observations from the using dataset that *did not* match observations from the master dataset.
- 3 = Observations from both datasets that did match.
- 4 = Observations from both datasets that did match, missing values in the master dataset were updated.
- 5 = Observations from both datasets that matched, values in the master dataset disagree with those in the using dataset.

Usually, you will want all the observations to have a value of 3. Values of 4 occur only when you use the "**update**" option. Values of 5 occur only when you use the "**update**" and "**replace**" options. If you need to merge another dataset, you have to re-name or drop **_merge** first, other wise you will get a "**_merge already defined**" error.

Collapse

"**collapse**" is used when you want to create a dataset containing the means sums, etc., of the various groups in the data. One example might be when you have one dataset of monthly data and another of yearly data and you need to analyze both sets of information together.

```
collapse (mean) var1 var2, by(month)
```

This will create a dataset of the means of var1 and var2 by month. If you have twelve months in your original dataset, then you will have twelve observations in the collapsed dataset. You must be careful, though, because Stata will compute the statistics on a variable-by-variable basis. If one variable has more missing observations than another,

the means (and any other statistics you request) will be based on a different number of observations. This is not always an acceptable practice. To avoid this, you must use the "**cw**" options for "casewise deletion." This means that Stata will drop any observation that does not have data for both var1 AND var2, thereby ensuring that all the statistics will be based on the same number of observations.

```
collapse (mean) var1 var2, by(month) cw
```

Basic Commands

Now that you have your data in a format you want, check it before doing any analyses. This can save you quite a bit of frustration later on.

Summarize

"**sum**", short for **summarize**, will give you the means, sd's, etc. of the variables listed. If you don't list any variables, it will give you the information for all numeric variables. If a variable you thought was numeric shows up as having 0 observations and a mean of 0, then, most likely, Stata still thinks it's a character variable.

```
summ  
summ var1 var2
```

The "**detail**" option gives you additional information about the distribution of the variable.

```
summ var1, detail
```

Inspect

"**inspect**" is another easy way to eyeball the distribution of a variable.

```
inspect var1
```

Tabulate

"**tab**", short for **tabulate**, will produce frequency tables. By specifying two variables, you will get a crosstab. There are other options to get the row, column and cell percentages as well as chi-square and other statistics; check the manual.

```
tab var1  
tab var1 var2  
tab var1 var2, row col chisq
```

"By" group processing

Sometimes you'll want to run a command or analysis on different groups of observations. The "*by variable*:" subcommand is the same thing as running the command with separate "if" statements for each group. You must sort the data before you can use the "by:"

```
sort sex
by sex: summ var2
by sex: gen lag2=var2[_n-1]
```

Sometimes, this can produce more output on the screen than you would care to look at. To suppress output to the screen, but not to the log file, use "**quietly**."

```
quietly by var1: gen lag2=var2[_n-1]
```

Fancy commands

Macro variables

Sometimes you need to use many variables the same way many times. One example would be if you want to run regressions on different dependent variables using the same set of independent variables. This can mean a lot of typing. One way around this is to create a macro variable. A macro variable is simply a variable that has as its value a particular string. This string can be anything you specify: a list of variables, a particular command, or whatever. Whenever Stata comes across the macro variable, it will interpret it to mean whatever string you set the variable to.

```
local macvar var1 var2 lagtot lag2
reg depvar `macvar'
```

In this example, when Stata "sees" the variable `macvar` in the regression command, it replaces ``macvar'` with the string "att att2 itt itt2 date." Pay careful attention to the different type of quotation marks: the first quote in ``macvar'` is the opening left quote usually found under the `~`. The second quote is the closing right, or single quote usually found under the `"`.

For

Other times, you will want to perform the same command on several variables. Again, this can mean a lot of typing. The "**for**:" command can also be very useful in these situations. It can use several types of variable lists, and with a little ingenuity, can be very powerful. Here are a few examples:

```
for var1-var25: replace @=. if @==99
for var*: replace @=. if @=99
for 1-3, ltype(numeric): gen q@=0
for a b c, ltype(any): gen str2 @="x"
```

The @ in the for command represents the variable names in the list, one by one. In the first two examples, Stata would interpret the command as though you had typed `replace var1=. if var1=99, replace var2=. if var2=99, and so on.`

In any command, you can specify two kinds of lists:

1. `var1-var25` or `date-q25` indicates all variables between `var1` and `var25`, inclusive, even if they do not begin with "var" as found when you run the `describe` command.
2. `var*` indicates all variables starting with "var."

in the third example, the command will create three new variables called `q1`, `q2`, and `q3` and sets them all equal to zero. The last example creates three new variables called `a`, `b`, and `c`, and sets them all equal to "x."

Regression

Linear Regression Model

In Stata you can estimate Linear Regression Models using the "**regress**" command (shortcut **reg**). Here we suppose that you are familiar with linear regression models, and more in general with multiple regression analysis, and thus we don't provide any introduction to this subject.

Now, suppose your dependent variable is *depvar* and your independent variables are listed in *varlist*. Then the command to run the linear regression is:

```
reg depvar varlist
```

The first variable after the **regress** command is always the dependent variable (or left-hand-side variable), and the following list gives the relevant independent variables (or right-hand-side variables).

In the food data set, you can try regression of food on income.

Suppose, for example, that you have a data set with individual observations on earnings, and you are interested in regressing the natural logarithm of average weekly earnings (*lnw*) on a sex dummy variable (*sex*: 0=male, 1=female), on a race dummy variable (*race*: 0=white, 1=non-white), on years of schooling (*school*), on age (*age*) and on a constant. Then you should type

```
reg lnw sex race school age
```

Notice that Stata automatically adds the "constant" (*_cons*) to the list of independent variables. If you want to exclude it, you can use the option **nocons**:

```
reg lnw sex race school age, nocons
```

As for many other commands in Stata, the **regress** command can be also applied to a subset of your data set by specifying an **if** (or **in**) statement. Suppose that you want to run a regression only for the male population:

```
reg lnw race school age if sex==0
reg lnw race school age if sex==0 & age>32
```

Several options are available with the **regress** command (see `help regress`). Among the most useful is the **robust** option, that allows you to run the regression using a "robust" estimator for the variance (specifically, the Huber/White/sandwich estimator).

This option is mainly used when the residuals are thought to be heteroschedastic:

```
reg lnw sex race school age, robust
```

Another option that may be useful is the one that allows you to run Instrumental Variable regressions (or 2SLS). Suppose you have a dependent variable $y1$ and a list of independent variables $x1$ $x2$ $x3$; and let's suppose that you want to instrument $x3$ with $z1$, $x1$ and $x2$. Then you can simply run the (2SLS) regression by typing:

```
reg y1 x1 x2 x3 (z1 x1 x2)
```

and you will get the results of the second stage in the 2SLS procedure.

Logit, Probit

When your dependent variable is dichotomous (zero-one), you can run a Linear Probability Model (using the [regress](#) command) or you might be interested in running a Logit or a Probit regression. Here we assume that you are familiar with the Logit and Probit models, as well as with maximum likelihood estimation. As a quick reminder, the only difference between the Linear Probability Model (LPM), the Logit model and the Probit model is the assumption that you make about the probability distribution of your dependent variable. Let y be your dependent variable (dummy: zero-one), and let x and z be your independent variables. Then the general specification in all these three models is $\text{Prob}(y = 1) = F(bx + cz)$, $\text{Prob}(y = 0) = 1 - F(bx + cz)$

where b and c are the coefficients of x and z , and $F(\cdot)$ is some function. The LPM assumes that $F(a) = a$. The Logit model assumes that $F(\cdot)$ is the logistic distribution function, while the Probit model assumes that $F(\cdot)$ to be the normal distribution function.

To run a Logit regression in Stata, you simply type:

```
logit depvar varlist
```

where *depvar* is your dependent (dichotomous) variable, and *varlist* is the list of independent variables that enter the model. To run a Probit model, the command is similar:

```
probit depvar varlist
```

Suppose that you are trying to estimate the effectiveness of a new drug being tested for cancer patients. Your dependent variable is *died* (1 if patient died, 0 otherwise), and your independent variables are *age* (age of patient at start of experiment) and *placebo* (1 if drug is placebo). Then your logit and probit estimates will be: .

Stata reports the maximum likelihood estimates for the original coefficients in both the **probit** and **logit** commands. You might be interested in other "coefficients". The **dprobit**

command estimates a Probit model, but reports the change in the probability for an infinitesimal change in each independent variable instead of the coefficient for each independent variable. The **logistic** command estimates a Logit model but reports the odds ratios for each independent variable instead of the coefficients. After both these commands, you can obtain the original coefficients by simply typing (respectively) **probit** or **logit**.

The **logistic** command also offers several "post-estimation" commands that can be used to evaluate the goodness of fit of the model. These commands are entered after the estimation is performed, and are always referred to the "last" **logistic** estimation performed. See `help logistic` for a list.

Finally, all the above commands (as in [regress](#)) can be applied only to a subset of the observations, can be run without constant, and can generate estimates using robust standard errors.

Prediction

After all estimation commands (i.e. [reg](#), [logistic](#), [logit](#), [dprobit](#), [probit](#)) several "predicted" values can be computed. The most important are the "predicted values for the dependent variable" and the "predicted residuals". In order to obtain the predicted values for the dependent variable for any model you have to generate a new variable (call it *yhat*) that associates to each observation its predicted value based on the coefficients obtained from the estimation and based on the particular values of the independent variables for that observation. Suppose that you run a linear regression and you want to estimate the predicted (ln) average weekly wage for each individual: then you type

```
reg lnw sex race school age
predict yhat
```

then a new variable called *yhat* will be generated by Stata (if such a variable already exists you will get an error message, and you will have to choose another name for the predicted values). In the same way you can build a new variable (call it *res*) with the predicted "residual" for each observation:

```
reg lnw sex race school age
predict ehat, residual
```

Notice that after the [logistic](#) command, several other "predictions" are possible using the **lpredict** command (see `help lpredict`).

Testing Linear Hypothesis

Here we assume that you are familiar with the theory of hypothesis testing. After each estimation, for each independent variable Stata automatically provides a t-test (for linear regressions) and a z-test (for Logit or Probit models) on the null hypothesis that the "true" coefficient is equal to zero. However you can perform several other tests on linear hypothesis about the coefficients. Suppose you run a linear regression of (log) average weekly earnings on sex, race, schooling and age. You could test the hypothesis that the coefficient on race is equal to -.05 (i.e. earnings for non-whites are 5% lower than for whites):

```
reg lnw sex race school age
test race=-.05
```

You could also be interested in testing hypotheses on several variables. For example you could be interested in testing the null hypothesis that the coefficients on sex and race are jointly equal to zero, or that they sum up to one:

```
reg lnw sex race school age
test sex race - tests the null that they are jointly equal to zero
test sex + race == 1
```